

Lecture 3: The likelihood of a POMP, and computing it with the particle filter

Qianying (Ruby) Lin Jesse Wheeler Spencer J. Fox

Objectives

By the end of this lecture you will be able to:

1. Say what the likelihood of a POMP model *is*, in plain language.
2. Explain why we cannot just simulate our way to it.
3. Run `pfilter` on your own laptop and get a log-likelihood for a fixed set of parameters.
4. Explain why that number is **random**, and what to do about it.
5. Read the filter's diagnostics, and know when you have enough particles.

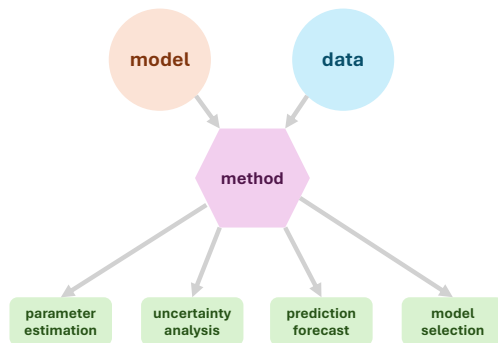
This lecture is deliberately light on formulas. The ideas here are geometric, and we are going to look at them rather than derive them.

The shape of the whole module

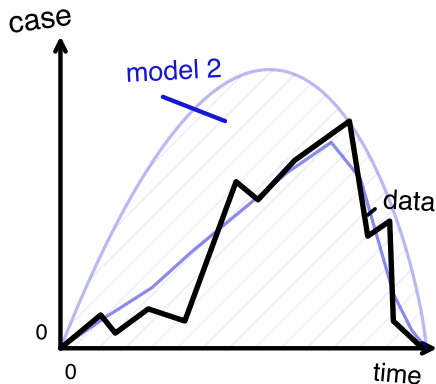
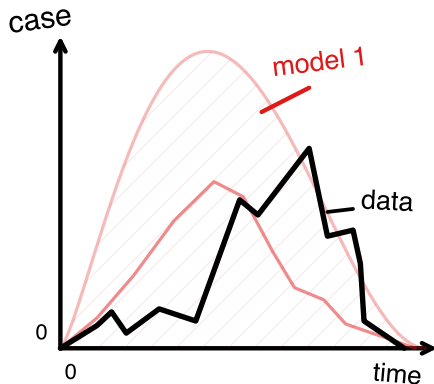
- ▶ We have a **model** and we have **data**.
- ▶ We need a **method** that connects them.
- ▶ Everything we want (parameter estimates, uncertainty, forecasts, model comparison) comes out the other side.

This session is about the method.

Specifically: about **scoring** a set of parameters.



The question, in one picture



Two models. One dataset. **Which model is doing a better job?**

Each shaded band is a model **simulated many times**. We turn that question into a *number*, the **likelihood**, and for stochastic models like these the only way to compute it is by **simulating**.

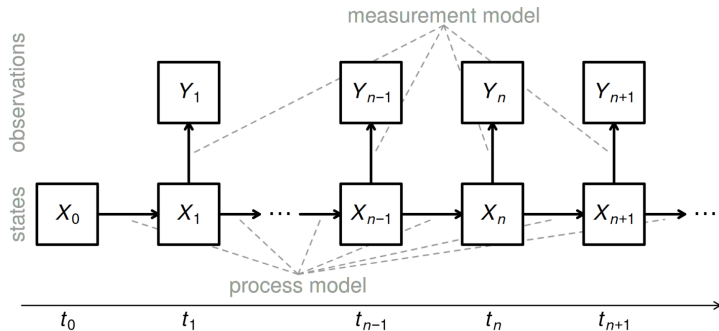
The likelihood, in one sentence

The likelihood is the probability that this model, with these parameters, would have produced exactly the data we actually saw.

- ▶ Bigger is better.
- ▶ A single likelihood means nothing on its own. It only means something **compared to another one**.
- ▶ We work with the **log**-likelihood, because the numbers involved are astronomically small.

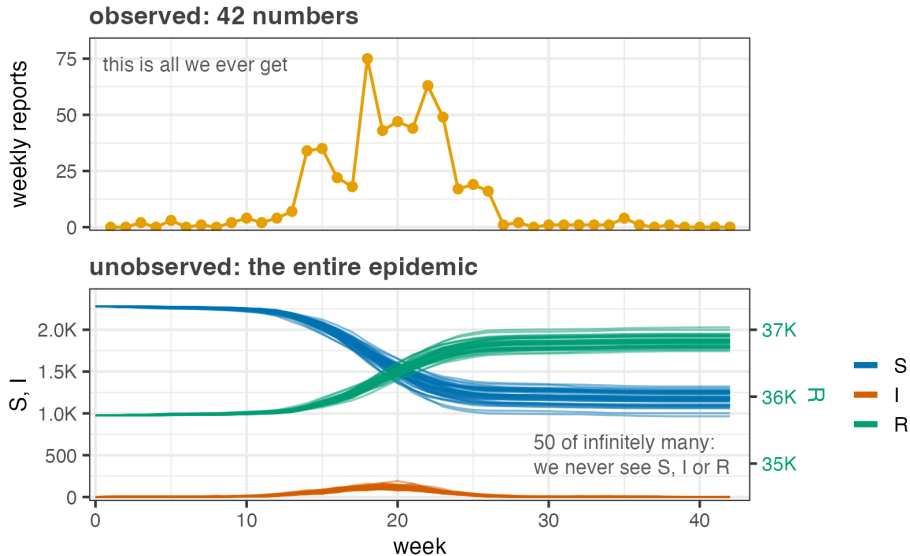
That is the whole definition. Everything from here is about how to *compute* it.

The problem: most of the epidemic is invisible



We observe the top row. We never see the bottom row: who is susceptible, who is infected.

The problem: most of the epidemic is invisible



So the likelihood is a hard sum

To score one parameter set honestly, we would have to ask:

*Across **every possible** hidden epidemic, weighted by how likely each one is: how plausible are my 42 numbers?*

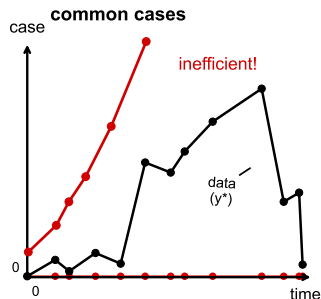
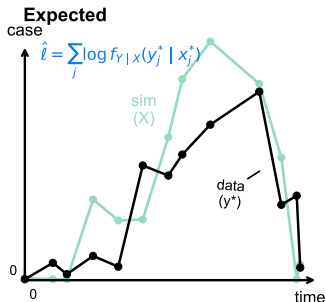
- ▶ That is a sum over an enormous space of possibilities.
- ▶ For our measles model there is **no formula** for it.
- ▶ But notice: we can **simulate** hidden epidemics all day long.

Simulating forward is easy. Scoring backward is hard.

That asymmetry is the reason this module exists.

The obvious idea

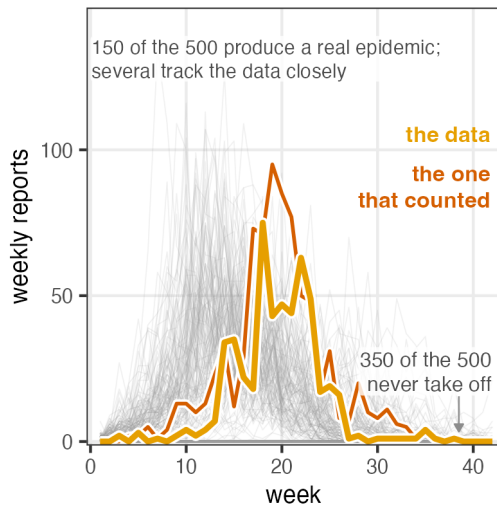
If we can simulate epidemics, why not simulate a *lot* of them and see which ones look like the data?



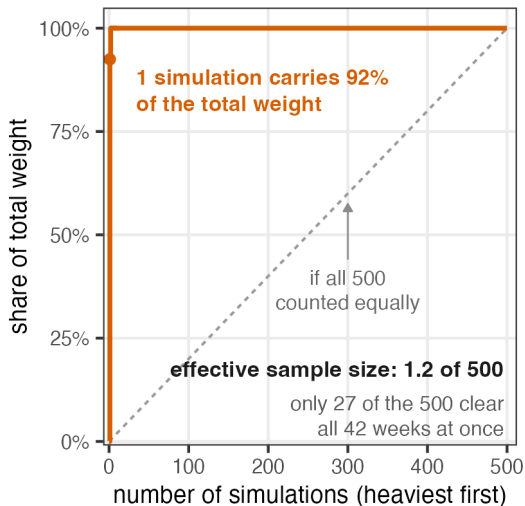
Simulate many trajectories, score each one against the data, average. This is a perfectly correct method. It is also useless.

Why it fails

500 simulations. Looks promising.



Essentially one of them counted.



Why it fails

- ▶ A simulation that drifts away from the data **never comes back**. Nothing pulls it home.
- ▶ Over 42 weeks, essentially every trajectory drifts away.
- ▶ So almost all the simulations contribute nothing, and the answer is decided by a **handful of lucky ones**.

The method is not *wrong*. It is just spending all its effort on epidemics that stopped resembling the data weeks ago.

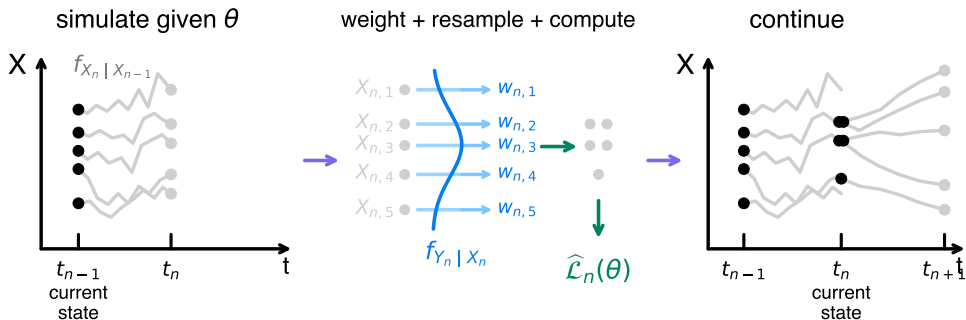
We need to stop bad trajectories early, not after 42 weeks.

The fix: don't let bad guesses run

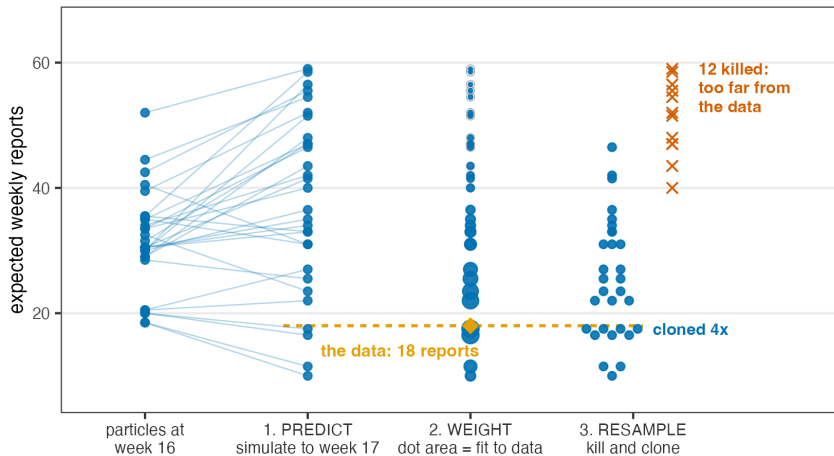
The particle filter changes one thing. Instead of letting a simulation run to the end, it checks in **at every observation**:

1. **Predict**: let every particle take one step forward.
2. **Weight**: the data arrive. Particles that predicted well score highly.
3. **Resample**: kill the bad ones, clone the good ones.

Then repeat, 42 times.



One week of the particle filter, on real data



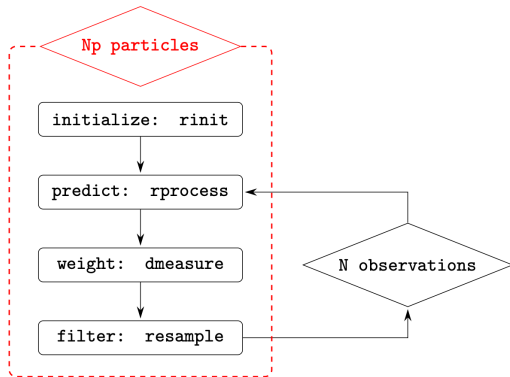
Direct simulation lets bad trajectories run for 42 weeks. The particle filter kills them **every single week** and clones the survivors.

What the algorithm needs from you

The particle filter only ever asks your model two things:

- ▶ *“Take a step forward.”*
(`rprocess`)
- ▶ *“How well does this state explain this week’s data?”* (`dmeasure`)

It never asks for a formula. That is why it works on models we cannot write down.



Doing it in pomp

```
source("scripts/model_measSIR.R")  
  
pf <- measSIR |> pfilter(Np=1000)  
logLik(pf)
```

That is the whole thing. N_p is the number of particles.

Exercises 1 and 2. Open 3_likelihood_pfilter.Rproj, then
scripts/EXERCISES.R.

Look at the model against the data, then compute your first log-likelihood.

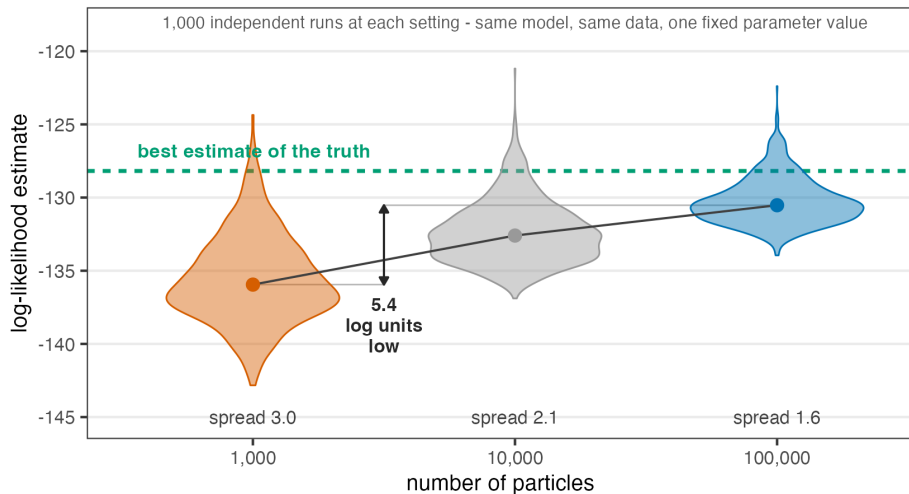
Run it twice, get two answers

The particle filter is a **Monte Carlo** algorithm. It does not return the likelihood. It returns a *random estimate* of it.

```
replicate(5, logLik(measSIR |> pfilter(Np=1000)))
```

Five runs, five different numbers. So: how many particles do we need, and how wrong are we if we skimp?

Too few particles doesn't just add noise



In practice: escalate $N_p = 2000$, then 5000, then 10000.

Too few particles doesn't just add noise

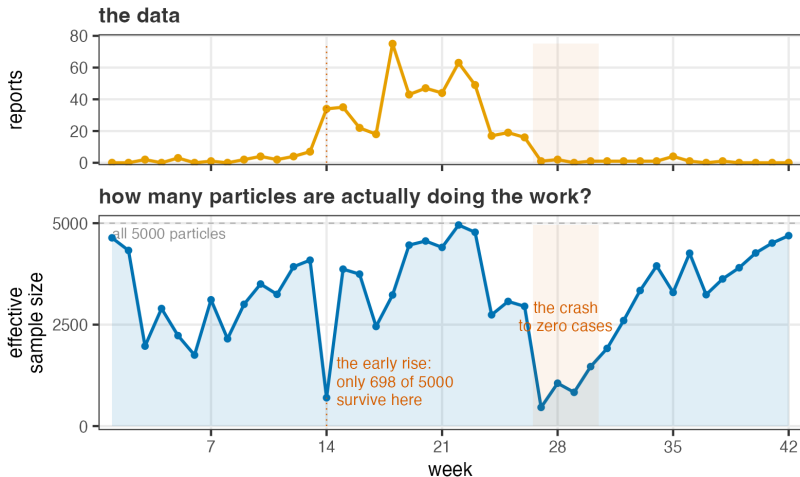
- ▶ More particles makes the cloud **tighter**; that much is expected.
- ▶ More particles also moves the cloud **up**.

With too few particles you do not get a noisy answer. You get a **confidently wrong** one, biased low.

Practical rule: if adding particles keeps raising your log-likelihood, you did not have enough particles.

Where does the noise come from? A quick diagnostic

pomp gives you `plot(pf)`. Its key panel is the **effective sample size**: at each week, how many of your particles are really contributing?



Where does the noise come from? A quick diagnostic

- ▶ Where ESS is high, hundreds or thousands of particles agree; that week's contribution to the likelihood is solid.
- ▶ Where ESS **crashes**, the whole week rests on a few hundred particles. That is where your estimate is fragile, and where the run-to-run noise is born.
- ▶ Here the filter struggles at the **early rise** and at the **crash to zero**, not at the big mid-epidemic jump, which it handles fine.

This is a *filter* health check: it tells you whether you have enough particles, and where. Using it to criticize the *model* comes later, in the workflow.

Exercises 3 and 4. See the number jump around, then read the diagnostic that explains why.

Combining repeated runs

Run several filters and combine them, but **not** with `mean()`:

```
pfs |> logLik() |> logmeanexp(se=TRUE)
```

Read the name backwards: `log(mean(exp(x)))`.

- ▶ The particle filter is unbiased for the **likelihood**.
- ▶ It is *not* unbiased for the **log**-likelihood.
- ▶ So we average on the likelihood scale, then take the log.

`logmeanexp` does this without overflowing, and gives you a standard error.

Each run is independent, so parallelize

```
library(foreach); library(doParallel); library(doRNG)

cl <- makePSOCKcluster(detectCores()-1)
registerDoParallel(cl)
registerDoRNG(1234)

foreach(i=1:20, .combine=c, .packages="pomp", .export="measSIR") %dopar% {
  measSIR |> pfilter(Np=5000)
} -> pfs

stopCluster(cl)
```

makePSOCKcluster works on macOS, Linux **and** Windows. registerDoRNG makes the parallel random numbers reproducible.

Exercises 5 and 6. Parallel evaluation, and what accuracy costs you.

The exercises at a glance

Everything builds to one idea: **the likelihood is a number we can compute, and it is random.**

1. **Look before you compute.** Simulate 20 outbreaks. Do they resemble the data?
2. **Your first likelihood.** One `pfilter`, one log-likelihood.
3. **Run it again.** The same line returns a *different* number. Why?
4. **Read the diagnostics.** Effective sample size week by week: where does the filter struggle?
5. **Run many, in parallel.** Combine 20 estimates with `logmeanexp` and a standard error.
6. **What accuracy costs.** Time `pfilter` against N_p : the price of precision.
7. **(stretch) A likelihood slice.** Vary η , watch the likelihood move: a first look at the surface.

Where we've got to

We can now **score any fixed set of parameters**:

- ▶ the particle filter turns a parameter vector into a log-likelihood;
- ▶ it only asks the model to simulate and to weigh, so it works even on models we cannot write down;
- ▶ the answer is random, so we repeat and combine with `logmeanexp`;
- ▶ more particles buys accuracy, not just precision.

But a score for *one* parameter set is not the answer. We want the set that fits the data **best**: the maximum-likelihood estimate.

After the break: which parameters maximize the likelihood, and the algorithm that finds them.

If you want to go deeper

You do not need any of this to do the exercises. It is here for the people who want the theory we skipped.

- ▶ **The pomp package**, and the framework behind everything in this module: King, Nguyen, and Ionides (2016).
- ▶ **The particle filter**: Arulampalam et al. (2002) is the standard tutorial; Kitagawa (1987) is the original; Doucet, Freitas, and Gordon (2001) is the reference volume.
- ▶ **The likelihood**, as a general tool for inference: Pawitan (2001).
- ▶ **Iterated filtering**, which is the next session: Ionides et al. (2015).

The maths we skipped is all in these, and the key formulas are in the supplement at the end of this deck.

References I

- Arulampalam, M. S., S. Maskell, N. Gordon, and T. Clapp. 2002. "A Tutorial on Particle Filters for Online Nonlinear, Non-Gaussian Bayesian Tracking." *IEEE Trans Signal Process* 50: 174–88. <https://doi.org/10.1109/78.978374>.
- Doucet, Arnaud, Nando de Freitas, and Neil Gordon, eds. 2001. *Sequential Monte Carlo Methods in Practice*. New York: Springer-Verlag.
- Ionides, Edward L., Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. 2015. "Inference for Dynamic and Latent Variable Models via Iterated, Perturbed Bayes Maps." *Proc Natl Acad Sci* 112 (3): 719–24. <https://doi.org/10.1073/pnas.1410597112>.
- King, Aaron A., Dao Nguyen, and Edward L. Ionides. 2016. "Statistical Inference for Partially Observed Markov Processes via the R Package Pomp." *J Stat Softw* 69 (12): 1–43. <https://doi.org/10.18637/jss.v069.i12>.
- Kitagawa, Genshiro. 1987. "Non-Gaussian State-Space Modeling of Nonstationary Time Series." *J Am Stat Assoc* 82 (400): 1032–41. <https://doi.org/10.1080/01621459.1987.10478534>.

References II

Pawitan, Yudi. 2001. *In All Likelihood: Statistical Modelling and Inference Using Likelihood*.
Oxford: Clarendon Press.

License, acknowledgments, and links

- ▶ This lesson is prepared for the Simulation-based Inference for Epidemiological Dynamics module at the Summer Institute in Statistics and Modeling in Infectious Diseases, SISMID.
- ▶ The materials build on previous versions of this course and related courses.
- ▶ Licensed under the Creative Commons Attribution-NonCommercial license. Please share and remix non-commercially, mentioning its origin. 
- ▶ Produced with R version 4.5.1 and pomp version 6.4.

R code for this lesson

The likelihood of a POMP

For reference: the same ideas as the lecture's pictures, written as the formulas from the original *likelihood* lesson. None of this is needed for the exercises.

The data are N observations $y_{1:N}^*$; the model is a density $f_{Y_{1:N}}(y_{1:N}; \theta)$. The **likelihood** evaluates it at the data, and the **log-likelihood** is its log:

$$\mathcal{L}(\theta) = f_{Y_{1:N}}(y_{1:N}^*; \theta), \quad \ell(\theta) = \log \mathcal{L}(\theta).$$

The Markov structure factors the joint density of states and observations into an **initial**, a **transition**, and a **measurement** density:

$$f_{X_{0:N}, Y_{1:N}}(x_{0:N}, y_{1:N}; \theta) = f_{X_0}(x_0; \theta) \prod_{n=1}^N f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta) f_{Y_n|X_n}(y_n | x_n; \theta).$$

The likelihood is an integral, and simulating it directly fails

The likelihood marginalizes over **every possible hidden path** $x_{0:N}$, a high-dimensional integral with no closed form:

$$\mathcal{L}(\theta) = \int f_{X_0}(x_0; \theta) \prod_{n=1}^N f_{Y_n|X_n}(y_n^* | x_n; \theta) f_{X_n|X_{n-1}}(x_n | x_{n-1}; \theta) dx_{0:N}.$$

Write it as an **expectation** over unconditional simulations $X_{0:N} \sim f_{X_{0:N}}$, then approximate by an average over J of them:

$$\mathcal{L}(\theta) = \mathbb{E} \left[\prod_{n=1}^N f_{Y_n|X_n}(y_n^* | X_n; \theta) \right] \approx \frac{1}{J} \sum_{j=1}^J \prod_{n=1}^N f_{Y_n|X_n}(y_n^* | X_n^j; \theta).$$

Correct, but simulated paths drift off the data, so the effort needed to keep any near it grows explosively with N . That is why we filter instead.

The particle filter I

Factor the likelihood **one observation at a time**:

$$\mathcal{L}(\theta) = \prod_{n=1}^N \int f_{Y_n|X_n}(y_n^* | x_n; \theta) f_{X_n|Y_{1:n-1}}(x_n | y_{1:n-1}^*; \theta) dx_n.$$

Two steps carry the particle cloud from one observation to the next. **Prediction** pushes it forward through the dynamics; **filtering** reweights it by the new observation, via Bayes:

$$f_{X_n|Y_{1:n-1}} = \int f_{X_n|X_{n-1}} f_{X_{n-1}|Y_{1:n-1}} dx_{n-1},$$
$$f_{X_n|Y_{1:n}} = \frac{f_{Y_n|X_n}(y_n^* | x_n; \theta) f_{X_n|Y_{1:n-1}}}{\int f_{Y_n|X_n}(y_n^* | u_n; \theta) f_{X_n|Y_{1:n-1}} du_n}.$$

The particle filter II

The filter estimates each factor by Monte Carlo, so the log-likelihood is a sum of per-observation pieces:

$$\ell(\theta) = \sum_{n=1}^N \log \mathcal{L}_n(\theta) \approx \sum_{n=1}^N \log \hat{\mathcal{L}}_n(\theta).$$

The estimate $\hat{\mathcal{L}}(\theta)$ is **unbiased** for $\mathcal{L}(\theta)$; taking logs makes $\log \hat{\mathcal{L}}(\theta)$ a consistent but **downward-biased** estimate of $\ell(\theta)$, which is why too few particles read low.